TECHNICAL ASSESSMENT SUBMISSION REPORT

Attendance Management Mobile Application

Production-Grade React Native, Expo SDK 58 & TypeScript Architecture

---

Role & Objective: Senior React Native Engineer, Software Tester & Systems Architect
Framework: React Native with Expo (SDK 58.0.17) & TypeScript 6.0
Persistence: AsyncStorage (Zero External Server Dependencies)
Authentication: Predefined Tamil / Indian Employee Profiles (Admin, Senior Staff, Staff)
Verification: 35/35 Automated Tests Passing • 20/20 Expo Doctor Checks Passing • 0 Lint Errors
Document Date: October 2026
Target Platform: Cross-Platform (Android & iOS)

Architecture	Test Suite	Code Quality
Role-Based (Admin, Senior, Staff)	35 Passed / 0 Failed (Jest)	Strict TypeScript & ESLint

Table of Contents

1. Executive Summary	3
2. Problem Statement & Objectives	3
3. Functional & Non-Functional Requirements	4
4. Technology Stack & Architectural Justifications	4
5. Application Architecture & Project Structure	5
6. TypeScript Data Models & Contract Definitions	6
7. Demonstration Accounts & Tamil/Indian Employee Directory	7
8. Authentication, Session Restoration & Persistence	8
9. Role-Based Permissions & Access Matrix	9
10. Navigation Architecture & Protected Stacks	10
11. Role-Specific Dashboard Implementations	11
12. Attendance Business Rules & Monotonic Validation	12
13. Attendance History, Search & Multi-Criteria Filtering	13
14. Admin User Management Directory	14
15. State Management & Offline AsyncStorage Pipeline	15
16. UI/UX Design System, Color Tokens & Components	16
17. Security Boundaries & Enterprise Limitations	17
18. Automated Testing Strategy & Verification Results	18
19. Installation, Startup & Execution Commands	19
20. Screen Recording Demonstration Script	20
21. Conclusion & Technical Deliverables Reference	20

1. Executive Summary

This project report details the engineering, implementation, and quality verification of the **Attendance Management Mobile Application**. Built using **React Native**, **Expo SDK 58**, and **TypeScript**, the solution delivers an enterprise-grade workforce monitoring platform that operates 100% offline without remote server dependencies. The application features strict role-based access control across three user tiers (Admin, Senior Staff, and Staff), enforceable business rules ensuring exactly one attendance record per employee per calendar day, real-time status transitions, multi-criteria filtering, and local persistence backed by AsyncStorage.

All employee accounts reflect authentic Indian and Tamil identities. Initial attendance records are generated dynamically using relative date offsets, ensuring that live dashboards, past histories, and actionable check-in/checkout buttons are immediately functional on first launch without hardcoded totals.

2. Problem Statement & Objectives

Problem Statement:

Modern corporate and field workforces require seamless, tamper-proof attendance recording with clear organizational hierarchy. Traditional mobile prototypes often suffer from state synchronization drift, race conditions caused by rapid double-tapping, data overwrites on application restarts, and client-side security leaks where supervisor portals display sensitive executive records.

Project Objectives:

- **Hierarchical Portals:** Provide three discrete user experiences: Organization-wide analytics for Admin, Staff-isolated team monitoring for Senior Staff, and personal self-service for Staff.
- **Deterministic Business Rules:** Automatically generate ISO-8601 timestamps, prevent duplicate check-ins, validate checkout against prior check-in, and reject checkout timestamps that precede check-in.
- **Offline Persistence:** Guarantee session continuity and attendance persistence across application restarts while preserving user modifications.
- **Rigorous Automated Testing:** Achieve 100% test passage across authentication, business logic, authorization, filtering, and storage using Jest.

3. Functional & Non-Functional Requirements

Category	Requirement Specification
Authentication	Secure login with empty-field validation, email syntax checking, demo account quick-fill chips, and persistent session restoration without storing passwords in session state.
Role Separation	Admin (universal visibility + user management), Senior Staff (strictly Staff visibility), Staff (own records only).
Attendance Logic	Single record per calendar day (YYYY-MM-DD), automatic timestamping, monotonic check-in to check-out progression, concurrency locking.
Analytics & Filters	Live KPI calculations from raw data, case-insensitive employee name search, date presets (Today, Yesterday, Week, Custom), newest-first sorting.
Reliability & QA	Zero TypeScript errors, zero ESLint warnings, 20/20 Expo Doctor diagnostics passing, 100% passing automated test suite.

4. Technology Stack & Architectural Justifications

Technology	Version	Architectural Rationale
React Native	0.88.0	High performance cross-platform native rendering across iOS and Android.
Expo SDK	58.0.17	Standardized managed workflow, verified native modules, seamless packaging.
React	19.3.0	Modern React 19 compiler optimization with rigorous hook purity guarantees.
TypeScript	6.0.3	Strict static typing preventing runtime null pointer and schema errors.
React Navigation	7.5.1	Native stack transitions with conditional role-protected screen stacks.
AsyncStorage	2.2.0	Asynchronous offline persistence with schema validation and seed guards.
Jest & RTL	30.5.2	Deterministic unit and integration testing with injectable mock clocks.

5. Application Architecture & Project Structure

The project follows a clean, modular architecture with clear physical separation between UI components, screens, navigation routers, state context, business services, data repositories, and types:

```
attendance-management-app/
■■■ App.tsx                     # Root entry point, SafeAreaProvider & AppProvider
■■■ package.json / tsconfig.json# Strict dependency and TypeScript configuration
■■■ jest.config.js              # Test configuration with jest-expo preset
■■■ src/
■   ■■■ components/             # Reusable UI library (AppHeader, CheckInOutCard, SummaryCard, et
c.)
■   ■■■ context/AppContext.tsx  # Central state management & lifecycle orchestration
■   ■■■ data/                   # Predefined Indian employee data & relative seed generator
■   ■■■ navigation/             # AppNavigator with role-protected stack screens
■   ■■■ screens/                # Auth, Admin, Senior Staff, Staff, Attendance & Profile screens
■   ■■■ services/               # Attendance business rules & AsyncStorage persistence
■   ■■■ theme/theme.ts          # Central design system tokens (colors, typography, spacing)
■   ■■■ types/index.ts          # TypeScript interfaces (User, AuthUser, AttendanceRecord)
■   ■■■ utils/                  # Date helpers with mockable clock & permission selectors
■■■ tests/                      # 5 test suites covering 35 unit/integration test cases
■■■ docs/                       # Comprehensive documentation, checklist & PDF report
```

6. TypeScript Data Models

Strong typing guarantees schema consistency across storage boundaries and UI layers. Crucially, the active session model (AuthUser) explicitly excludes plaintext passwords:

```
// User Roles & Attendance Statuses
export type UserRole = 'admin' | 'senior_staff' | 'staff';
export type AttendanceStatus = 'not_checked_in' | 'checked_in' | 'checked_out';

// Active Authenticated Session (Passwords strictly omitted)
export interface AuthUser {
  id: string;
  name: string;
  email: string;
  role: UserRole;
  department?: string;
}

// Attendance Entity (Linked via stable foreign key userId)
export interface AttendanceRecord {
  id: string;
  userId: string;
  date: string;               // ISO format: YYYY-MM-DD
  checkInTime: string | null;  // ISO-8601 timestamp string
  checkOutTime: string | null; // ISO-8601 timestamp string
  status: AttendanceStatus;
}
```

7. Demonstration Accounts & Tamil/Indian Employee Directory

In strict compliance with Requirement 5, all predefined dummy employees reflect authentic Indian and Tamil names. The system initializes with 12 distinct employees covering 1 Admin, 2 Senior Staff members, and 9 Staff members:

User ID	Name	Role	Email	Password	Department
USR001	Ram Kumar	Admin	admin@test.com	Admin@123	Executive Admin
USR002	Nivin Pauly	Senior Staff	senior@test.com	Senior@123	Engineering Ops
USR003	Arjun Kumar	Staff	staff@test.com	Staff@123	Mobile Engineering
USR004	Priya Lakshmi	Senior Staff	priya@test.com	Priya@123	QA & Delivery
USR005	Karthik Raj	Staff	karthik@test.com	Karthik@123	Frontend Dev
USR006	Divya Shree	Staff	divya@test.com	Divya@123	UI/UX Design
USR007	Vijay Kumar	Staff	vijay@test.com	Vijay@123	Backend Services
USR008	Meena Devi	Staff	meena@test.com	Meena@123	Technical Ops
USR009	Suresh Kumar	Staff	suresh@test.com	Suresh@123	Data Infra
USR010	Anitha Rani	Staff	anitha@test.com	Anitha@123	Customer Support
USR011	Hari Prasad	Staff	hari@test.com	Hari@123	DevOps & Cloud
USR012	Kavitha M	Staff	kavitha@test.com	Kavitha@123	Quality Eng.

8. Authentication & Session Persistence Workflow

The login screen features input sanitization, regular expression email verification, show/hide password toggling, error banners, and convenience quick-fill chips for Admin, Senior Staff, and Staff. Upon credential verification against DUMMY_USERS:

1. An AuthUser entity is created with the user's ID, Name, Email, Role, and Department. The password string is intentionally dropped.
2. The entity is serialized to JSON and persisted to AsyncStorage key @attendance_app_session_v1.
3. React state in AppContext is updated, causing AppNavigator to transition automatically to the role's dashboard.
4. On app cold starts, getSession() reads the key, verifies object integrity via a type predicate, and restores the session immediately without requiring re-login.

9. Role-Based Permissions & Access Matrix

Access control is implemented in src/utils/permissions.ts and applied to both navigation routers and data selectors. This ensures that Senior Staff cannot view Admin records even if they manipulate query parameters:

Feature / Capability	Admin	Senior Staff	Staff
Access Admin Dashboard	**YES**	NO	NO
Access Senior Staff Dashboard	NO	**YES**	NO
Access Staff Dashboard	NO	NO	**YES**
View Organization-Wide Attendance	**YES (All Roles)**	NO	NO
View Staff Team Attendance	YES	**YES (Staff Only)**	NO
Access User Directory Management	**YES**	NO	NO
View Any Employee's History	**YES**	**YES (Staff Only)**	NO (Self Only)
Perform Personal Check-In / Check-Out	YES	YES	YES
View Personal History Log	YES	YES	YES
Sign Out & Reset Navigation	YES	YES	YES

10. Navigation Architecture & Protected Stacks

Using React Navigation Native Stack, the navigation hierarchy mounts conditional screens based directly on the authenticated user's role:

- **Unauthenticated (currentUser === null):** Mounts LoginScreen only. All protected screens are unmounted.
- **Admin Stack:** Mounts AdminDashboardScreen, TeamAttendanceScreen, UserManagementScreen, PersonalHistoryScreen, ProfileScreen.
- **Senior Staff Stack:** Mounts SeniorDashboardScreen, TeamAttendanceScreen, PersonalHistoryScreen, ProfileScreen.
- **Staff Stack:** Mounts StaffDashboardScreen, PersonalHistoryScreen, ProfileScreen.
- **Logout Navigation Reset:** Discarding the active user session automatically unmounts all protected routes and mounts the Login screen, permanently preventing back-button access into authenticated views.

11. Role-Specific Dashboard Implementations

Admin Dashboard: Displays current date, organization-wide KPIs (Total Employees: 12, Staff: 9, Senior Staff: 2, Checked In Today, Pending), quick action navigation grid, personal check-in/out widget, and recent cross-departmental attendance activity.

Senior Staff Dashboard: Strictly isolated to Staff oversight. Displays Staff Team metrics (Total Staff: 9, Staff Checked In, Staff Pending), today's Staff-only activity list, personal check-in/out widget, and direct links to Staff Attendance and Personal History.

Staff Dashboard: Designed for fast employee self-service. Features employee profile banner, Today's Status card, live timestamps, the primary Check-In / Check-Out action button, and a shortcut to personal history.

12. Attendance Business Rules & Validation Workflow

The attendance engine (src/services/attendanceService.ts) enforces strict rules governing check-in and checkout operations:

Operation	Validation Gate & Business Invariant
Check-In	1. Authenticate active session. 2. Retrieve current calendar date in local ISO YYYY-MM-DD format. 3. Verify user does not already have an attendance record for today (reject duplicate check-ins). 4. Generate ISO-8601 timestamp automatically. 5. Set status to checked_in with null checkOutTime. 6. Await AsyncStorage write before updating UI context.
Check-Out	1. Locate today's existing attendance record for user. 2. Reject checkout if no check-in record exists. 3. Reject duplicate checkout if already checked_out. 4. Generate checkout ISO-8601 timestamp. 5. Validate that checkout time is not earlier than check-in time. 6. Update status to checked_out and persist.
Concurrency Lock	A module-scoped lock set (pendingOperations) blocks rapid repeated taps from spawning concurrent check-in or checkout mutations.
Mockable Clock	Injectable time helper (setMockDate) enables exact time advancement in automated tests.

13. Attendance History, Search & Filtering

The reusable Team Attendance screen provides rich filtering options for supervisors: Case-insensitive name search, quick date presets (All, Today, Yesterday, Past 7 Days), an interactive custom date modal, role filter tabs for Admin, and a reset button. Records are sorted newest-first by date descending and check-in time descending.

14. Admin User Management Directory

The User Management screen is restricted to Admin. It lists all 12 predefined employees with name, email, employee ID, department, contact number, and color-coded role badges. Admins can search by name or email, filter by role (All, Admin, Senior Staff, Staff), and tap any employee to drill into their full attendance history. Passwords are never displayed.

15. State Management & Storage Pipeline

State is managed via the React Context API (AppContext.tsx). On startup, initializeAppData() inspects local storage. If storage is empty, dynamic seed records relative to today are written. If already initialized, existing records are preserved. All state writes are persisted to AsyncStorage before React state transitions occur.

16. Automated Testing Strategy & Verification Results

Automated tests were developed using **Jest** with the jest-expo preset and an in-memory AsyncStorage mock. All 5 test suites and 35 individual test cases passed completely with zero failures:

Test Suite File	Scope / Module	Tests	Status
tests/auth/login.test.ts	Mandatory accounts (Admin, Senior, Staff), empty fields, format validation, bad password	7 Passed	**PASS**
tests/services/storageService.test.ts	Session persistence, logout clearing, corrupt recovery, seed idempotency	5 Passed	**PASS**
tests/services/attendanceService.test.ts	Check-in, checkout, duplicate checks, timestamps, 1 per day, multi-user independence	10 Passed	**PASS**
tests/utils/permissions.test.ts	Role restrictions, Admin all-access, Senior Staff staff-only isolation, navigation guards	6 Passed	**PASS**
tests/screens/filtering.test.ts	Case-insensitive search, date filters, combined filters, latest-first sort, null format	7 Passed	**PASS**
TOTALS	**Comprehensive Coverage across All Requirements**	**35 Passed**	**100%**

17. Quality Gates & Diagnostics Summary

Validation Tool	Command Executed	Result & Findings
Jest Test Runner	`npm test`	35 passed, 35 total (2.029s execution time)
TypeScript Compiler	`npm run typecheck`	0 errors, strict mode enabled with Jest types
Expo ESLint	`npm run lint`	0 errors, 0 warnings across all project files
Expo Doctor Diagnostics	`npx expo-doctor`	20/20 checks passed. No issues detected

18. Installation, Execution & Command Reference

1. Installation:

```
npm install
```

2. Start Development Server:

```
npx expo start
```

3. Run on Android Emulator:

```
npx expo start --android
```

4. Run on iOS Simulator: (Requires macOS and Xcode)

```
npx expo start --ios
```

5. Run Test Suite:

```
npm test # or: npm run test:ci
```

6. Typecheck and Lint:

```
npm run typecheck
npm run lint
```

19. Known Limitations & Enterprise Roadmap

- **Local Client RBAC Boundary:** Client-side permission checks govern mobile rendering. In enterprise production, an authenticated API gateway and server-side row-level security must enforce permissions.
- **Unencrypted Local Storage:** AsyncStorage stores data as unencrypted JSON. For production compliance (HIPAA/GDPR), sensitive tokens should reside in expo-secure-store.
- **Geofencing & Biometrics:** Future iterations may integrate expo-location for geofenced office check-ins and expo-local-authentication for fingerprint/FaceID check-in authorization.

20. Appendix: Demonstration Credentials

Admin Account: admin@test.com / Admin@123 (User ID: USR001 - Ram Kumar)
Senior Staff Account: senior@test.com / Senior@123 (User ID: USR002 - Nivin Pauly)
Staff Account: staff@test.com / Staff@123 (User ID: USR003 - Arjun Kumar)

Report Generated: October 2026 • Verified against running Expo SDK 58 application codebase.